

Fundamentos de Sistemas Robóticos com ROS 2

Etapa 6

Professor Mario Pego

mario.peggo@prof.infnet.edu.br



Cronograma



Fundamentos de Sistemas Robóticos com ROS 2

<i>Etapas</i>	<i>Segunda-feira/Quarta-feira</i>
Etapa 1	20/07/26 e 22/07/26
Etapa 2/Leitura TP1	27/07/26 e 29/07/26
Etapa 3	03/08/26 e 05/08/26
Etapa 4/Entrega do TP1/Leitura TP2	10/08/26 e 12/08/2026
Etapa 5	17/08/26 e 19/08/26
Etapa 6/Entrega TP2/Leitura do TP3	24/08/26 e 26/08/26
Etapa 7	31/08/26 e 02/09/26
Etapa 8/Entrega TP3	07/09/26 (FERIADO) e 09/09/26
Etapa 9/Leitura do AT	14/09/26 e 16/09/26
Entrega Assessment	21/09/26 e 23/09/26
Reentrega	28/09/26 e 30/09/26

Implementar ações ROS 2 para
execução assíncrona de
objetivos de longa duração

O que vamos ver

1

Ações: goal, feedback, result

O terceiro mecanismo de comunicação do ROS 2

2

example_interfaces/action/Fibonacci

Reaproveitando uma interface pronta em vez de criar uma nova

3

Action Server com MultiThreadedExecutor

Por que tarefas longas exigem execução paralela

4

patrulha_server.py e patrulha_client.py

O par completo do TP3 Exercício 3

O que já temos: tópicos e serviços

Das Etapas 3 a 5 — dois dos três mecanismos de comunicação do ROS 2

- Tópicos (Etapas 3-5): fluxo contínuo, muitos-para-muitos, sem confirmação — `velocidade_pub`, `estado_robo`
- Serviços (Etapas 3-5): pergunta-resposta síncrona, um-para-um — `modo_server`, controlador
- Nenhum dos dois resolve bem uma tarefa que demora (segundos a minutos) e precisa relatar progresso no meio do caminho
- É exatamente esse o problema que o terceiro mecanismo do ROS 2 — ações — resolve

Por que ações existem

Tarefas longas precisam de progresso e de cancelamento

- Um serviço bloqueia o client até a resposta final — inaceitável se a tarefa demora 30 segundos
- Um tópico não tem noção de 'início' e 'fim' de uma tarefa específica, nem devolve um resultado
- Uma ação combina os três: um goal inicia a tarefa, feedback relata progresso ao longo do tempo, result entrega o resultado final
- Exemplos reais: navegar até um ponto, executar uma sequência de movimentos, varrer uma área — e, hoje, patrulhar N pontos

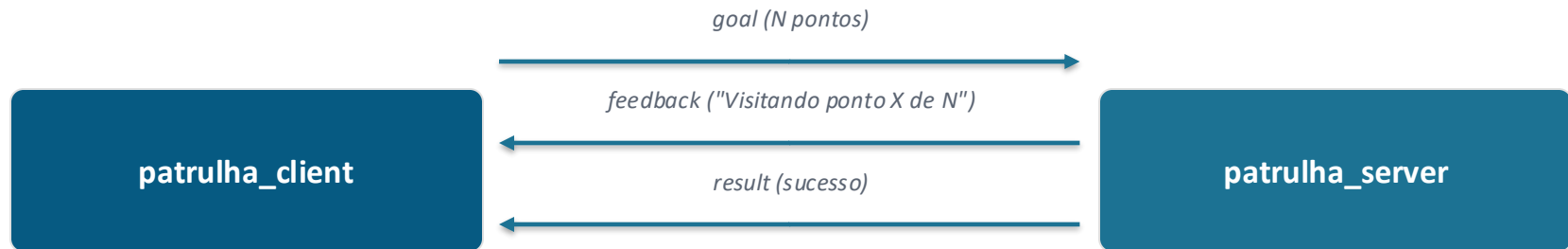
As três fases de uma ação

goal → feedback (0 ou mais vezes) → result

- goal: o client envia UMA vez o que quer que seja feito (aqui, o número de pontos da rota)
- feedback: o server envia VÁRIAS vezes, ao longo da execução, informando o progresso
- result: o server envia UMA vez, ao final, com o resultado — sucesso, falha ou cancelamento
- Diferente de um serviço, o client não fica bloqueado esperando: ele recebe feedback continuamente e só o result encerra a ação

Quem envia o quê, em qual direção

Client inicia; Server relata progresso e conclui



Só o goal parte do client. Feedback e result sempre partem do server — é o oposto do fluxo de um serviço, onde o client é quem espera passivamente.

Por que o Action Server precisa de MultiThreadedExecutor

`execute_callback` fica "ocupado" durante toda a tarefa

- `execute_callback` roda por vários segundos (aqui, 1s por ponto da rota) — ele não retorna até a tarefa terminar
- Com `SingleThreadedExecutor`, nada mais roda nesse tempo: nem um pedido de cancelamento, nem outro goal
- Com `MultiThreadedExecutor`, o node continua respondendo a outros callbacks (cancelamento, novos goals) enquanto `execute_callback` ainda está executando
- É o mesmo raciocínio do controlador.py na Etapa 5 — qualquer callback que 'demora' precisa rodar sem travar o resto do nó

Reaproveitando example_interfaces/action/Fibonacci

O TP3 pede para usar essa interface pronta como base estrutural

```
$ ros2 interface show \  
    example_interfaces/action/Fibonacci  
  
int32 order  
---  
int32[] sequence  
---  
int32[] sequence
```

Nosso mapeamento

order = número de pontos da rota (o goal).

Result e Feedback usam o MESMO nome de campo, sequence — só o tipo muda:

Feedback.sequence é a lista parcial (cresce a cada 1s), Result.sequence é a lista final.

Nenhum campo precisa ser criado, só reinterpretado.

Anatomia do `execute_callback`

O coração do Action Server — roda uma vez por goal aceito

```
def execute_callback(self, goal_handle):  
    for i in range(1, n + 1):  
        if goal_handle.is_cancel_requested:  
            goal_handle.canceled()  
            return Result()  
        # ... trabalho de verdade aqui  
        goal_handle.publish_feedback(fb)  
    goal_handle.succeed()  
    return Result()
```

Três chamadas obrigatórias

`is_cancel_requested` (chechar a cada iteração), `publish_feedback` (relatar progresso) e `succeed()/canceled()` (fechar a ação com um estado final). Esqueceu `succeed()`? A ação nunca termina do ponto de vista do client.

Anatomia do lado client

Três callbacks encadeados, nenhum bloqueante

```
future = client.send_goal_async(  
    goal, feedback_callback=cb_feedback)  
future.add_done_callback(cb_goal_response)  
  
# cb_goal_response recebe o goal_handle  
# e dispara get_result_async()  
  
# cb_feedback roda a cada feedback recebido  
# cb_result roda uma vez, no final
```

Tudo assíncrono

Nenhuma dessas chamadas bloqueia — `send_goal_async` retorna na hora, e os callbacks disparam sozinhos conforme o server envia feedback e, por fim, o result. É por isso que o client também precisa de `rclpy.spin()`.

patrulha_server.py — parte 1 de 3

Imports e construtor: registra o Action Server

salvar em `comunicacao_ros2/comunicacao_ros2/patrulha_server.py`

```
import time
import rclpy
from rclpy.action import ActionServer
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node
from example_interfaces.action import Fibonacci
class PatrulhaServer(Node):
    def __init__(self):
        super().__init__('patrulha_server')
        self._action_server = ActionServer(
            self, Fibonacci, 'patrulha', self.execute_callback)
        self.get_logger().info('nó iniciado')
```

patrulha_server.py — parte 2 de 3

execute_callback: visita cada ponto, com feedback e cancelamento

salvar em `comunicacao_ros2/comunicacao_ros2/patrulha_server.py`

```
def execute_callback(self, goal_handle):
    n = goal_handle.request.order
    feedback_msg = Fibonacci.Feedback()
    feedback_msg.sequence = []
    for i in range(1, n + 1):
        if goal_handle.is_cancel_requested:
            goal_handle.canceled()
            return Fibonacci.Result()
        time.sleep(1.0)
        feedback_msg.sequence.append(i)
        self.get_logger().info(f'Visitando ponto {i} de {n}')
        goal_handle.publish_feedback(feedback_msg)
    goal_handle.succeed()
    result = Fibonacci.Result()
    result.sequence = feedback_msg.sequence
    return result
```

patrulha_server.py — parte 3 de 3

main() com MultiThreadedExecutor

salvar em `comunicacao_ros2/comunicacao_ros2/patrulha_server.py`

```
def main(args=None):
    rclpy.init(args=args)
    node = PatrulhaServer()
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    try:
        executor.spin()
    finally:
        executor.shutdown()
        node.destroy_node()
        rclpy.shutdown()
```

patrulha_client.py — parte 1 de 3

Imports, construtor e send_goal

salvar em `comunicacao_ros2/comunicacao_ros2/patrulha_client.py`

```
import sys
import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
from example_interfaces.action import Fibonacci

class PatrulhaClient(Node):
    def __init__(self):
        super().__init__('patrulha_client')
        self._client = ActionClient(self, Fibonacci, 'patrulha')

    def send_goal(self, n):
        self.n = n
        goal_msg = Fibonacci.Goal()
        goal_msg.order = n
        self._client.wait_for_server()
        future = self._client.send_goal_async(
            goal_msg, feedback_callback=self.feedback_callback)
        future.add_done_callback(self.goal_response_callback)
```


patrulha_client.py — parte 2 de 3

Os dois callbacks: resposta do goal e feedback

salvar em `comunicacao_ros2/comunicacao_ros2/patrulha_client.py`

```
def goal_response_callback(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().info('Goal rejeitado')
        return

    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.get_result_callback)

def feedback_callback(self, feedback_msg):
    n_visitados = len(feedback_msg.feedback.sequence)
    self.get_logger().info(f'Visitando ponto {n_visitados} de {self.n}')
```

patrulha_client.py — parte 3 de 3

get_result_callback e o main()

salvar em `comunicacao_ros2/comunicacao_ros2/patrulha_client.py`

```
def get_result_callback(self, future):  
    result = future.result().result  
    self.get_logger().info(f'Resultado: {result.sequence}')
```



```
def main(args=None):  
    rclpy.init(args=args)  
    client = PatrulhaClient()  
    client.send_goal(int(sys.argv[1]))  
    rclpy.spin(client)
```

Registrando os dois entry_points

patrulha_server e patrulha_client entram juntos no setup.py

salvar em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        # ... velocidade_pub, monitor_velocidade,
        # ... modo_server, modo_client,
        # ... controlador, estado_robo
        'patrulha_server = '
        'comunicacao_ros2.patrulha_server:main',
        'patrulha_client = '
        'comunicacao_ros2.patrulha_client:main',
    ],
},
```

Sétimo e oitavo executáveis

Server e client vivem no mesmo pacote, mas rodam como processos separados — cada um com seu próprio entry_point.

Compilando e subindo o patrulha_server

colcon build de novo — dois novos entry_points registrados

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2  
$ source install/setup.bash  
$ ros2 run comunicacao_ros2 patrulha_server
```

```
[INFO] [patrulha_server]: nó iniciado
```

Deixe rodando

Mantenha este terminal aberto — é este processo que aguarda o goal do patrulha_client no próximo passo.

Enviando um goal: patrulhar 3 pontos

Terminal do server (já rodando) e terminal do client, lado a lado

Terminal 1 — patrulha_server

(já rodando, aguardando goals)

[INFO] [patrulha_server]:

Visitando ponto 1 de 3

Visitando ponto 2 de 3

Visitando ponto 3 de 3

Terminal 2 — patrulha_client

```
$ ros2 run comunicacao_ros2 \
  patrulha_client -- 3
```

Visitando ponto 1 de 3

Visitando ponto 2 de 3

Visitando ponto 3 de 3

~3 segundos no total

Um segundo entre cada ponto (time.sleep(1.0) no server) — 'Visitando ponto 3 de 3' é seguido do result, e o processo client termina sozinho.

Conferindo a ação registrada

ros2 action list — critério de validação do TP3

```
$ ros2 action list  
/patrulha
```

No TP3

O Exercício 3 pede exatamente esse print, com o Server ativo — cole no relatório junto com a saída completa do goal com N=3.

Ações sem armadilhas comuns

- Sempre chame `goal_handle.succeed()` OU `goal_handle.canceled()` — sem isso, o client fica esperando um result que nunca chega
- Cheque `is_cancel_requested` dentro do loop de trabalho, não só no início do `execute_callback`
- `publish_feedback()` não bloqueia — pode ser chamado quantas vezes forem necessárias durante a execução
- Rode sempre o Action Server com `MultiThreadedExecutor` — `SingleThreadedExecutor` trava cancelamento e novos goals durante a execução

Tópico, serviço ou ação — como escolher

Situação	Mecanismo	Por quê
Dado contínuo, sem resposta necessária	Tópico	Publish/subscribe, muitos-para-muitos
Pergunta pontual, resposta rápida	Serviço	Request/response síncrono, um-para-um
Tarefa longa, com progresso e cancelamento	Ação	goal + feedback (N vezes) + result

OK

`goal / feedback / result`

três fases e quem envia cada uma

OK

`example_interfaces/action/Fibonacci`

interface pronta em vez de criar uma nova

OK

`patrulha_server.py +
patrulha_client.py`

Action Server e Client completos, com cancelamento tratado

OK

`MultiThreadedExecutor` no Action Server

é obrigatório aqui

Próximo passo

Com `patrulha_server.py` e `patrulha_client.py` prontos (Exercício 3 do TP3 coberto), falta juntar tudo: na próxima competência, vamos construir o `coordenador_vigilancia.py`, que integra tópicos, serviço e o estado da patrulha em um único nó orientado a eventos — o Exercício 4, que fecha o TP3.

Estruturar sistemas de
controle modular baseados
em eventos, mensagens e
ações distribuídas

O que vamos ver

1

Por que combinar os três mecanismos

Sistemas reais não usam só tópicos, só serviços ou só ações

2

Projetando a arquitetura de vigilância

Quatro componentes, cada um com o mecanismo certo

3

Flag interno e reação a eventos

Como um nó guarda estado e reage a dois gatilhos diferentes

4

`coordenador_vigilancia.py`

O nó completo do TP3 Exercício 4 — e o TP3 inteiro fechado

O que já sabemos proteger e paralelizar

Da Competência 1 da Etapa 5

- `threading.Lock` protege um atributo compartilhado entre callbacks (`controlador.py`)
- `MultiThreadedExecutor` permite que serviço, timer e subscriber rodem em paralelo no mesmo nó
- Hoje juntamos isso a um terceiro ingrediente: reagir a um tópico de sensor E a um serviço de comando, ao mesmo tempo, no mesmo nó

Sistemas reais combinam tópicos, serviços e ações

Cada mecanismo resolve um tipo de comunicação diferente

- Dado contínuo de sensor (distância) → tópico, porque chega o tempo todo e ninguém precisa confirmar recebimento
- Comando pontual (ligar/desligar alarme) → serviço, porque é uma pergunta com resposta imediata
- Tarefa longa (patrulhar N pontos) → ação, como vimos na Competência 1
- Um sistema de vigilância real precisa dos três ao mesmo tempo — é isso que o `coordenador_vigilancia.py` demonstra

Arquitetura do sistema de vigilância

Os quatro componentes pedidos pelo TP3 Exercício 4

Componente	Mecanismo	Mensagem
Leitura de distância frontal (contínua)	Tópico /distancia_frontal	sensor_msgs/msg/Range
Ligar/desligar alarme	Serviço /ligar_alarme	std_srvs/srv/SetBool
Patrulhar rota de N pontos	Ação /patrulha (Competência 1)	example_interfaces/action/Fibonacci
Status geral do sistema	Tópico /status_patrulha	std_msgs/msg/String

coordenador_vigilancia: quatro conexões, um nó

Duas assinaturas, um serviço exposto, uma publicação



/status_patrolha ainda não tem um publisher dedicado neste TP — a assinatura fica pronta para integração futura, mesma lógica do */bateria* na Etapa 5.

Anatomia do construtor

Duas assinaturas, um serviço, um publisher — tudo no `__init__`

```
self.alarme_ativo = False
self.create_subscription(
    Range, '/distancia_frontal', ...)
self.create_subscription(
    String, '/status_patrolha', ...)
self.create_service(
    SetBool, '/ligar_alarme', ...)
self.create_publisher(String, '/alertas', 10)
```

Um flag simples

`self.alarme_ativo` é um bool comum — não precisa de `threading.Lock` porque só é escrito por UM callback (o do serviço) e lido por outro (o de distância), nunca dois callbacks escrevendo nele ao mesmo tempo.

O serviço /ligar_alarme e o flag interno

SetBool controlando um estado que outro callback vai ler depois

```
def ligar_alarme_callback(self, req, resp):  
    self.alarme_ativo = req.data  
    resp.success = True  
    resp.message = (  
        'Alarme ativado' if req.data  
        else 'Alarme desativado')  
    return resp
```

Mesmo padrão da Etapa 5

É a mesma ideia do mudar_estado_callback do controlador.py: o serviço não faz o trabalho pesado, só atualiza um flag que outro callback vai consultar.

A condição que dispara o alerta

Só publica em /alertas quando as duas condições são verdadeiras

```
def distancia_callback(self, msg):  
    if msg.range < 0.5 and \  
        self.alarme_ativo:  
        alerta = String()  
        alerta.data = 'INTRUSO DETECTADO'  
        self.pub_alertas.publish(alerta)
```

Duas condições, não uma

Distância crítica SEM alarme ativo não gera alerta — é o comportamento esperado de um sistema que o operador pode desligar. O 'and' é o coração da lógica deste exercício.

coordenador_vigilancia.py — parte 1 de 4

Imports e início do construtor: assinaturas

salvar em `comunicacao_ros2/comunicacao_ros2/coordenador_vigilancia.py`

```
import rclpy
from rclpy.node import Node
from rclpy.executors import MultiThreadedExecutor
from sensor_msgs.msg import Range
from std_msgs.msg import String
from std_srvs.srv import SetBool
class CoordenadorVigilancia(Node):
    def __init__(self):
        super().__init__('coordenador_vigilancia')
        self.alarme_ativo = False
        self.sub_dist = self.create_subscription(
            Range, '/distancia_frontal', self.distancia_callback, 10)
        self.sub_status = self.create_subscription(
            String, '/status_patrolha', self.status_callback, 10)
```

coordenador_vigilancia.py — parte 2 de 4

Fim do construtor: serviço, publisher e log de início

salvar em `comunicacao_ros2/comunicacao_ros2/coordenador_vigilancia.py`

```
self.srv_alarme = self.create_service(  
    SetBool, '/ligar_alarme', self.ligar_alarme_callback)  
self.pub_alertas = self.create_publisher(  
    String, '/alertas', 10)  
self.get_logger().info('nó iniciado')  
  
def ligar_alarme_callback(self, request, response):  
    self.alarme_ativo = request.data  
    response.success = True  
    estado = 'ativado' if self.alarme_ativo else 'desativado'  
    response.message = f'Alarme {estado}'  
    return response
```

coordenador_vigilancia.py — parte 3 de 4

distancia_callback e status_callback

salvar em `comunicacao_ros2/comunicacao_ros2/coordenador_vigilancia.py`

```
def distancia_callback(self, msg):  
    if msg.range < 0.5 and self.alarme_ativo:  
        alerta = String()  
        alerta.data = 'INTRUSO DETECTADO'  
        self.pub_alertas.publish(alerta)  
        self.get_logger().warn('INTRUSO DETECTADO')  
  
def status_callback(self, msg):  
    self.get_logger().info(f'status_patrolha: {msg.data}')
```

coordenador_vigilancia.py — parte 4 de 4

main() com MultiThreadedExecutor

salvar em `comunicacao_ros2/comunicacao_ros2/coordenador_vigilancia.py`

```
def main(args=None):
    rclpy.init(args=args)
    node = CoordenadorVigilancia()
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    try:
        executor.spin()
    finally:
        executor.shutdown()
        node.destroy_node()
        rclpy.shutdown()
```

Registrando o entry_point

coordenador_vigilancia precisa entrar no setup.py

salvar em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        # ... controlador, estado_robo,
        # ... patrulha_server, patrulha_client
        'coordenador_vigilancia = '
        'comunicacao_ros2.coordenador_vigilancia:main',
    ],
},
```

Nono executável

Com este, o pacote comunicacao_ros2 reúne todos os mecanismos de comunicação do ROS 2 vistos no TP2 e no TP3.

Compilando e subindo o coordenador

colcon build + ros2 run

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2  
$ source install/setup.bash  
$ ros2 run comunicacao_ros2 coordenador_vigilancia
```

```
[INFO] [coordenador_vigilancia]: nó iniciado
```

Deixe rodando

Mantenha este terminal aberto — os próximos dois passos usam outros terminais para acionar o alarme e simular a distância.

Ativando o alarme

ros2 service call — sintaxe exata do TP3 (YAML com chaves)

```
$ ros2 service call /ligar_alarme \  
    std_srvs/srv/SetBool "{data: true}"  
  
response:  
std_srvs.srv.SetBool_Response(  
    success=True, message='Alarme ativado')
```

Duas sintaxes válidas

'data: true' (aspas simples) e "{data: true}" (chaves, aspas duplas) são a mesma coisa em YAML — o TP3 usa a segunda forma para este exercício.

Simulando uma distância crítica

Terminal 1: publica a distância. Terminal 2: observa o alerta

Terminal 1 — ros2 topic pub

```
$ ros2 topic pub --once \  
  /distancia_frontal \  
  sensor_msgs/msg/Range \  
  "{range: 0.3}"
```

Terminal 2 — ros2 topic echo

```
$ ros2 topic echo /alertas  
  
data: INTRUSO DETECTADO  
---
```

--once é seu, não do TP3

O enunciado mostra só 'ros2 topic pub ...'; sem --once o comando publica em loop e trava o terminal. --once envia uma mensagem e devolve o prompt — mesmo resultado, mais prático.

Coordenadores orientados a eventos

- Cada callback deve ser curto — se precisar de tempo (como o `patrulha_server`), isso pertence a uma ação, não a um callback de tópico ou serviço
- Flags de estado só precisam de Lock quando MAIS DE UM callback pode escrever neles — aqui, só `ligar_alarme_callback` escreve em `alarme_ativo`
- Prefira condições explícitas ('and' com as duas checagens) a lógica implícita — facilita auditoria, que o TP3 cobra na rubrica final
- `MultiThreadedExecutor` é o padrão para qualquer nó que combine tópico + serviço, com ou sem ação envolvida

0 TP3 completo, exercício por exercício

Exercício	Arquivo	Mecanismo(s)
1 — Estado do robô	estado_robo.py (Etapa 5)	4 publishers, mensagens padrão
2 — Controlador	controlador.py (Etapa 5)	Serviço + Lock + MultiThreadedExecutor
3 — Patrulha	patrulha_server/client.py (Etapa 6)	Ação (goal/feedback/result)
4 — Vigilância	coordenador_vigilancia.py (Etapa 6)	Tópico + serviço + flag interno

OK

Arquitetura de 4 componentes

mecanismo escolhido para cada um

OK

`/ligar_alarme (SetBool)`

controla um flag interno lido por outro callback

OK

`/distancia_frontal → /alertas`

condição dupla (distância E alarme) disparando o alerta

OK

TP3 completo

os quatro exercícios devem ter arquivo, entry_point e validação

TP3 completo — próximo destino: o Assessment

Com `estado_robo.py`, `controlador.py`, `patrulha_server/client.py` e `coordenador_vigilancia.py` prontos, os quatro exercícios do TP3 estão cobertos. A partir da Etapa 7, entramos no Assessment (AT): parametrização e arquitetura (Etapa 7), URDF do robô hospitalar (Etapa 8) e simulação no Gazebo com Launch files (Etapa 9).